

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2014

Huấn luyện viên: Hu Weidong

China Computer Federation, 2014

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2014

IOI China candidate team papers / National training team 2014 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2014. Phần mục lục và đầu sách có lớp văn bản đọc được, nhưng thân bài dùng ảnh xạ phông chữ khiến kết quả trích xuất bị biến dạng. Bản dịch này chuyển ngữ phần đầu sách, mục lục và ba bài đầu tiên bằng cách kết hợp mục lục trích xuất được với OCR từ các trang đã render.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2014*. Huấn luyện viên ghi trên bìa là Hu Weidong. Đơn vị xuất bản là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Báo cáo ra đề <i>MSS</i>	Wang Ziyu, Trường Trung học số 1 Shengli, Đông Doanh, Sơn Đông
9	Báo cáo ra đề <i>Matrix</i>	Yu Xingjiang, Trường Trường Quận, Trường Sa, Hồ Nam
19	Đa giác biến đổi	Dong Honghua, Trường Trung học số 1 Thiệu Hưng, Chiết Giang
35	Bước đầu nghiên cứu thuật toán liên quan tới nhóm hoán vị	Cen Ruoxu, Trường Trung học Trần Hải, Chiết Giang
45	Bàn về phụ thuộc tuyến tính	Kuang Zhengfei, Trường Yali, Trường Sa, Hồ Nam
57	Một số nghiên cứu về cây khung tích nhỏ nhất ba chiều	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng, Chiết Giang
65	Bàn về bài toán xâu con palindrome	Xu Yi, Trường Trung học cao cấp Thường Châu, Giang Tô
77	Bàn về ứng dụng phương pháp duy trì mảng nhiều chiều trong bài cấu trúc dữ liệu	Liang Zeyu, Trường Trung học số 1 Hợp Phì, An Huy
91	Ứng dụng cây đoạn trong một lớp bài chia để trị	Xu Yinzhan, Trường Học Quân Hàng Châu, Chiết Giang
105	Thuật toán căn bậc hai: không chỉ là chia khối	Wang Yuetong, Trường Ngoại ngữ Nam Kinh, Giang Tô
115	Bàn về các vấn đề liên quan tới cây động và một số mở rộng đơn giản	Huang Zhi'ao, Trường Yali, Trường Sa, Hồ Nam
137	Ứng dụng thuật toán ngẫu nhiên trong thi tin học	Hu Zecong, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam
155	Hiện thực chương trình tính tế: bàn về tối ưu hằng số trong OI	He Qi, Trường Trung học Nam Khai, Thiên Tân
169	Trở về gốc rễ: phép toán bit và ứng dụng	Shen Yang, Trường Trung học Trần Hải, Chiết Giang
195	Một số phương pháp tìm nghiệm tốt thứ k	Yu Dingli, Trường Trung học số 1 Thiệu Hưng, Chiết Giang

3 Báo cáo ra đề *MSS*

Wang Ziyu, Trường Trung học số 1 Shengli, Đông Doanh

3.1 Mô tả bài toán

3.1.1 Phát biểu

Trên mặt phẳng có N điểm. Mỗi điểm có một trọng số; ban đầu mỗi điểm thuộc một tập khác nhau. Gọi tập hiện tại thứ i là S_i . Cần duy trì một cấu trúc dữ liệu hỗ trợ bốn loại thao tác sau.

- Merge $x\ y$: chuyển mọi điểm trong tập S_x sang tập S_y .
- Split $i\ d\ v$, với $d \in \{0, 1\}$: tạo hai tập mới S_{c+1} và S_{c+2} , trong đó c là số tập hiện có, kể cả các tập rỗng sinh ra trước đó. Sau đó, trong tập S_i , các điểm có tọa độ chiều thứ d không vượt quá v được chuyển vào S_{c+1} , còn các điểm có tọa độ lớn hơn v được chuyển vào S_{c+2} .
- Query i : hỏi giá trị lớn nhất, nhỏ nhất và tổng trọng số của các điểm trong tập S_i .
- Add $i\ d$: cộng d vào trọng số của mọi điểm trong tập S_i .

Để thuận tiện, với mỗi $d \in \{0, 1\}$, mọi điểm trong dữ liệu vào có tọa độ chiều thứ d đôi một khác nhau. Mọi tập được nhắc tới trong các thao tác đều không rỗng.

3.1.2 Dữ liệu vào

Dòng đầu chứa một số nguyên, biểu thị số điểm. Tiếp theo là N dòng, mỗi dòng gồm ba số nguyên lần lượt là hai tọa độ và trọng số của một điểm. Sau đó là một số nguyên Q , biểu thị số thao tác. Tiếp theo là Q dòng, mỗi dòng mô tả một thao tác theo định dạng ở trên.

3.1.3 Dữ liệu ra

Với mỗi thao tác Query, in ra ba số nguyên, lần lượt là trọng số lớn nhất, trọng số nhỏ nhất và tổng trọng số trong tập được hỏi.

3.1.4 Ví dụ

Dữ liệu vào

```
5
4 7 3
18 16 2
14 13 1
10 2 10
3 8 5
11
Merge 2 3
Merge 4 5
Split 2 1 13
Query 4
Query 6
Add 6 2
Query 6
Merge 6 4
Split 6 0 10
```

Query 9
Split 8 0 3

Dữ liệu ra

10 5 15
1 1 1
3 3 3
3 3 3

3.1.5 Quy mô dữ liệu và ràng buộc

Dữ liệu được chia thành năm nhóm:

- nhóm A chiếm 10%: $N, Q \leq 500$;
- nhóm B chiếm 20%: dữ liệu không có thao tác tách;
- nhóm C chiếm 10%: sau khi thao tác tách đầu tiên xuất hiện thì không còn thao tác gộp;
- nhóm D chiếm 20%: các điểm thỏa một quan hệ đặc biệt khiến bài toán suy biến về một chiều;
- nhóm E không có đặc trưng bổ sung.

Ở các nhóm B đến E, $N \leq 50000$, $Q \leq 100000$. Giới hạn thời gian là 3 giây và giới hạn bộ nhớ là 512 MB. Dòng ràng buộc của nhóm D trong bản OCR khá nhiều; phần lời giải phía sau cho thấy nhóm này là trường hợp các điểm có thứ tự một chiều tương thích, nên có thể chuyển mọi thao tác tách về cùng một trục.

3.2 Phân tích bài toán

3.2.1 Các thuật toán đơn giản

Mô phỏng. Với nhóm A, có thể mô phỏng trực tiếp các tập điểm. Độ phức tạp thời gian là $O(NQ)$.

Gộp theo heuristic. Với nhóm B, vì không có thao tác tách, có thể dùng gộp theo kích thước: mỗi tập được duy trì bằng một cây cân bằng. Khi gộp hai tập, ta chèn toàn bộ điểm của tập nhỏ hơn vào cây cân bằng của tập lớn hơn. Mỗi lần một điểm bị chèn lại, kích thước tập chứa nó ít nhất tăng gấp đôi; do đó mỗi điểm bị chèn lại nhiều nhất $O(\log N)$ lần. Độ phức tạp là $O(N \log^2 N + Q)$.

Tách theo heuristic. Với nhóm C, trước hết dùng gộp theo heuristic để xử lý toàn bộ thao tác gộp trước khi thao tác tách đầu tiên xuất hiện. Sau đó, nếu mỗi thao tác tách có thể thực hiện trong thời gian

$$O(T \cdot |\text{tập nhỏ hơn sau khi tách}|),$$

trong đó T không phụ thuộc vào kích thước tập nhỏ hơn, thì phân tích ngược lại giống hệt gộp theo heuristic.

Nếu mỗi lần chỉ tách theo một tọa độ, ta chỉ cần lập cây cân bằng theo thứ tự tọa độ đó. Quay lại bài này, với mỗi tập ta lập hai cây cân bằng, lần lượt sắp theo hoành độ và tung độ. Khi tách, dùng cây theo chiều được hỏi để lấy tập nhỏ hơn; trong cây còn lại, xóa các điểm tương ứng rồi dựng lại. Khi đó $T = O(\log N)$, nên độ phức tạp là $O(N \log^2 N + Q)$.

3.2.2 Thuật toán một chiều

Trong nhóm D, vì các điểm có thứ tự tương thích theo một chiều, mọi phép tách theo tung độ có thể chuyển thành phép tách theo hoành độ. Do đó có thể xem bài toán là duy trì các tập số trên một đường thẳng.

Dựa trên cây cân bằng. Ta tiếp tục thử dùng cây cân bằng để duy trì giá trị trong từng tập. Cây cân bằng hỗ trợ tách trong $O(\log N)$ và gộp trong $O(\log N)$ khi hai tập có khoảng giá trị không giao nhau; vấn đề còn lại là gộp khi các khoảng giá trị có thể xen kẽ nhau.

Thuật toán gộp có thể viết như sau.

```
merge(x, y):
  if x = nil: return y
  if y = nil: return x
  if x.min > y.min: swap(x, y)
  return merge(merge(splitL(x, y.min), y), splitR(x, y.min))
```

Ở đây $\text{merge}(a, b)$ trả về hợp của hai cây cân bằng a, b khi phạm vi giá trị của chúng không giao nhau; $\text{splitL}(a, v)$ trả về cây gồm các phần tử của a có giá trị không vượt quá v ; $\text{splitR}(a, v)$ trả về cây gồm các phần tử của a có giá trị lớn hơn v .

Ta xét độ phức tạp của thuật toán. Cần chứng minh rằng tổng chi phí của Q_1 thao tác tách và Q_2 thao tác gộp là

$$O((N + Q_1) \log^2 N).$$

Định nghĩa

$$W(X, Y) = \sum_{a \in X, b \in Y} [|\text{rank}(X \cup Y, a) - \text{rank}(X \cup Y, b)| = 1],$$

trong đó $\text{rank}(S, x)$ là thứ hạng theo giá trị của x trong S . Nói không hoàn toàn hình thức, $W(X, Y)$ là số đoạn, trừ đi 1, thu được sau khi gộp X và Y rồi nén các phần tử vốn thuộc cùng một tập thành một đoạn. Một thao tác gộp có chi phí $O(W(X, Y) \log N)$.

Gọi U là hợp của tất cả các tập. Với một phần tử x thuộc tập S , đặt

$$\begin{aligned} g_0(x) &= \text{rank}(U, x) - \text{rank}(U, \text{prev}(S, x)), \\ g_1(x) &= \text{rank}(U, \text{succ}(S, x)) - \text{rank}(U, x), \end{aligned}$$

trong đó $\text{prev}(S, x)$ và $\text{succ}(S, x)$ lần lượt là phần tử trước và sau của x trong tập S ; nếu không tồn tại thì dùng $\min(U)$ hoặc $\max(U)$.

Sau thao tác thứ i , định nghĩa thế năng của cấu trúc dữ liệu là

$$\Phi_i = c \log N \sum_{x \in U} (\log g_0(x) + \log g_1(x)),$$

với c là hằng số sẽ chọn sau. Ban đầu $\Phi_0 = cN \log^2 N$.

Một thao tác tách có chi phí thực $O(\log N)$. Vì chỉ có một giá trị g_0 và một giá trị g_1 tăng, phần tăng thế năng không vượt quá $2c \log^2 N$.

Xét thao tác gộp X và Y . Đặt

$$\begin{aligned} P_X &= \{x \in X \mid \text{prev}(X, x) \neq \text{prev}(X \cup Y, x)\} \cup \{\min(X)\}, \\ S_X &= \{x \in X \mid \text{succ}(X, x) \neq \text{succ}(X \cup Y, x)\} \cup \{\max(X)\}. \end{aligned}$$

Sau khi gộp X, Y và nén các điểm vốn thuộc cùng một tập thành đoạn, P_X chứa mọi đầu trái của các đoạn đến từ X , còn S_X chứa mọi đầu phải. Định nghĩa tương tự P_Y, S_Y . Khi đó

$$|P_X| + |P_Y| = W(X, Y) + 1.$$

Với các cặp biên kề nhau sau khi gộp, trong hai đại lượng g_0 và g_1 sẽ có một đại lượng giảm ít nhất còn một nửa giá trị cũ. Vì có $W(X, Y) - 1$ cặp như vậy, phần biến thiên thế năng do gộp không vượt quá

$$-c(W(X, Y) - 1) \log N.$$

Chọn c đủ lớn, tổng chi phí mọi thao tác được chặn bởi

$$\begin{aligned} \Phi_0 + \sum_{i=1}^{Q_1} O(\log N + c \log^2 N) \\ + \sum_{i=1}^{Q_2} (-c(W(X_i, Y_i) - 1) \log N + O(W(X_i, Y_i) \log N)) \\ = O((Q_1 + N) \log^2 N). \end{aligned}$$

Dựa trên cây đoạn. Một cách khác là dùng cây đoạn động để duy trì từng tập. Cây đoạn động hỗ trợ gộp; sau khi rời rạc hóa tọa độ, thao tác tách tốn $O(\log N)$. Cần chứng minh tổng chi phí của Q_1 thao tác tách và Q_2 thao tác gộp là

$$O((N + Q_1) \log N).$$

Gọi $\text{size}(T)$ là số nút của cây đoạn động biểu diễn tập T . Chi phí gộp hai tập X, Y là

$$O(\text{size}(X) + \text{size}(Y) - \text{size}(X \cup Y)).$$

Định nghĩa thế năng sau thao tác thứ i :

$$\Phi_i = c \sum_{S \in \mathcal{S}_i} \text{size}(S),$$

trong đó $\mathcal{S}_i$ là họ các tập hiện tại. Một thao tác tách có chi phí thực $O(\log N)$ và chỉ làm tăng thế năng thêm $O(c \log N)$. Một thao tác gộp X, Y làm thế năng giảm

$$c(\text{size}(X) + \text{size}(Y) - \text{size}(X \cup Y)).$$

Thế năng ban đầu là $cN \log N$. Chọn c đủ lớn, tổng chi phí không vượt quá

$$\begin{aligned} \Phi_0 + \sum_{i=1}^{Q_1} (O(\log N) + c \log N) \\ + \sum_{i=1}^{Q_2} (O(\text{size}(X_i) + \text{size}(Y_i) - \text{size}(X_i \cup Y_i)) \\ - c(\text{size}(X_i) + \text{size}(Y_i) - \text{size}(X_i \cup Y_i))) \\ = O((N + Q) \log N). \end{aligned}$$

Từ đó nhóm D được giải với độ phức tạp $O((N + Q) \log N)$.

Thuật toán thay thế. Đây là một bài toán kinh điển. Tài liệu tham khảo [3] đưa ra thuật toán trung bình $O(\log N)$ dựa trên *biased skip list*, nhưng cả phân tích lẫn hiện thực đều phức tạp hơn; độc giả quan tâm có thể tự tham khảo.

3.2.3 Thuật toán tổng quát

Xét phạm vi áp dụng của thuật toán một chiều. Định nghĩa quan hệ thứ tự riêng phần trên tập điểm:

$$(x_1, y_1) < (x_2, y_2) \quad \text{khi và chỉ khi} \quad x_1 < x_2, y_1 < y_2.$$

Nếu hợp của mọi tập hiện tại U , theo quan hệ này, là một chuỗi, thì thuật toán một chiều áp dụng được: do y tăng đơn điệu theo x , mọi phép chia theo tọa độ y đều có thể chuyển thành phép chia theo tọa độ x . Tương tự, khi U là một phản chuỗi, tức y giảm đơn điệu theo x , thuật toán một chiều cũng áp dụng được.

Trong trường hợp tổng quát, chia U thành một số tập con $U_1, \dots, U_K$ sao cho theo thứ tự riêng phần ở trên, mỗi U_i là một chuỗi hoặc phản chuỗi. Với mỗi U_i , xét giao của nó với từng tập hiện tại:

$$\{S \cap U_i \mid S \in \mathcal{S}\}.$$

Khi đó các thao tác gộp, tách và hỏi đều có thể xử lý độc lập trên từng U_i , và mỗi U_i được duy trì bằng thuật toán một chiều.

Độ phức tạp của cách này là $O((N + Q)K \log N)$. Còn cần chứng minh $K = O(\sqrt{N})$.

Bổ đề 1. Trong một tập có thứ tự riêng phần P kích thước N , hoặc tồn tại một chuỗi kích thước $\sqrt{N}$, hoặc tồn tại một phản chuỗi kích thước $\sqrt{N}$.

Chứng minh. Giả sử P không chứa chuỗi độ dài L . Theo định lý Dilworth, P có thể được chia thành không quá $L - 1$ phản chuỗi. Vì vậy tồn tại một phản chuỗi có kích thước ít nhất $N/(L - 1)$. Chọn $L = \lceil \sqrt{N} \rceil$ là đủ.

Bổ đề 2. Một tập có thứ tự riêng phần U kích thước N có thể được chia thành $O(\sqrt{N})$ chuỗi hoặc phản chuỗi.

Chứng minh. Chia đệ quy: mỗi lần xóa khỏi tập còn lại một chuỗi dài nhất hoặc một phản chuỗi dài nhất. Nếu tập còn lại có kích thước n , theo bổ đề 1 có thể xóa ít nhất $\sqrt{n}$ phần tử. Số phần trong phép chia thỏa

$$T(N) = T(N - \sqrt{N}) + 1 < T(N/2) + O(\sqrt{N}) = O(\sqrt{N}).$$

Như vậy bài toán được giải với độ phức tạp

$$O((N + Q)\sqrt{N} \log N).$$

3.3 Kết luận

Kiến thức dùng trong bài chỉ gồm cấu trúc dữ liệu cơ bản và phân tích khấu hao, đều thuộc phạm vi của thí sinh NOI. Tuy nhiên, việc tìm ra thuật toán tổng quát đòi hỏi năng lực phân tích khá tốt.

Đối với 40 điểm cuối, tuy còn tồn tại những cách giải khác, tác giả cho rằng cách chia tập có thứ tự riêng phần được trình bày ở đây thú vị và giàu tính gợi mở hơn. Khi đối tượng trong bài toán thỏa một quan hệ thứ tự riêng phần khác, chẳng hạn quan hệ bao hàm giữa các đoạn, những phương pháp tương tự đáng được cân nhắc.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
2. Richard A. Brualdi, *Introductory Combinatorics*, 5th ed., Prentice Hall.
3. John Iacono, Ozgur Ozkan, “Mergeable Dictionaries”.

4 Báo cáo ra đề *Matrix*

Yu Xingjiang, Trường Trường Quận, Trường Sa

4.1 Đề bài

4.1.1 Mô tả

Cho một ma trận số thực không âm A kích thước $N \times N$. Cần tìm một ma trận số thực B kích thước $N \times N$ sao cho

$$\sum_{k=1}^N B_{i,k} + \sum_{k=1}^N B_{k,j} \geq A_{i,j}$$

với mọi i, j . Nói cách khác, với mỗi ô (i, j) , tổng các phần tử trên hàng i của B cộng với tổng các phần tử trên cột j của B phải không nhỏ hơn $A_{i,j}$. Trong điều kiện đó, hãy tối thiểu hóa tổng mọi phần tử của B .

4.1.2 Dữ liệu vào

Có hai nhiệm vụ, chi tiết xem ở phần phạm vi dữ liệu. Với mỗi nhiệm vụ, dòng đầu chứa một số nguyên N . Tiếp theo là N dòng, mỗi dòng chứa N số thực không âm, mô tả ma trận A đã cho.

4.1.3 Dữ liệu ra

In hai dòng. Mỗi dòng in một số thực không âm giữ lại 4 chữ số sau dấu thập phân, lần lượt là đáp án của hai nhiệm vụ.

4.1.4 Ví dụ

Dữ liệu vào

```
3
1 7 3
3 2 1
5 4 1
2
1 1
1 1
```

Dữ liệu ra

```
6.5000
1.0000
```

4.1.5 Nhiệm vụ con và quy ước

- Task0: mọi phần tử trong A bằng nhau.
- Task1: trong cả nhiệm vụ một và nhiệm vụ hai, $N \leq 50$.
- Task2: các phần tử của A là ngẫu nhiên.
- Task3: trong nhiệm vụ hai, $N \leq 50$.
- Task4: trong nhiệm vụ một, $N \leq 300$; trong nhiệm vụ hai, $N \leq 80$.

Ở nhiệm vụ một, ma trận B cần dựng không có ràng buộc nào khác. Ở nhiệm vụ hai, mọi phần tử của B phải không âm. Mọi số thực trong dữ liệu vào giữ lại 3 chữ số sau dấu thập phân và có trị tuyệt đối không vượt quá 1000.

Giới hạn thời gian là 3 giây, giới hạn bộ nhớ là 512 MB.

4.2 Ý tưởng giải

4.2.1 Task0

Dễ thấy khi mọi phần tử của A bằng nhau, mọi phần tử trong một nghiệm tối ưu của B cũng có thể chọn bằng nhau. Do đó có thể tính trực tiếp đáp án bằng tay.

4.2.2 Lập mô hình quy hoạch tuyến tính

Theo đề bài, mô hình quy hoạch tuyến tính trực tiếp là

$$\begin{cases} \sum_{i=1}^N B_{i,1} + \sum_{j=1}^N B_{1,j} \geq A_{1,1}, \\ \sum_{i=1}^N B_{i,2} + \sum_{j=1}^N B_{1,j} \geq A_{1,2}, \\ \dots \\ \sum_{i=1}^N B_{i,1} + \sum_{j=1}^N B_{2,j} \geq A_{2,1}, \\ \sum_{i=1}^N B_{i,2} + \sum_{j=1}^N B_{2,j} \geq A_{2,2}, \\ \dots \\ \sum_{i=1}^N B_{i,N} + \sum_{j=1}^N B_{N,j} \geq A_{N,N}, \end{cases} \quad \min z = \sum_{i=1}^N \sum_{j=1}^N B_{i,j}.$$

Cách lập này có $O(N^2)$ biến, cần tối ưu thêm. Đề bài thực ra đã gợi ý rằng có thể dùng tổng hàng và tổng cột làm biến, nên ta thử hướng đó. Gọi R_i là tổng trọng số của hàng i , C_j là tổng trọng số của cột j . Khi đó có một mô hình mới:

$$\begin{cases} R_1 + C_1 \geq A_{1,1}, \\ R_1 + C_2 \geq A_{1,2}, \\ \dots \\ R_2 + C_1 \geq A_{2,1}, \\ R_2 + C_2 \geq A_{2,2}, \\ \dots \\ R_N + C_N \geq A_{N,N}, \\ \sum_{i=1}^N R_i - \sum_{j=1}^N C_j = 0. \end{cases}$$

Trong bài gốc, hàm mục tiêu của mô hình này được viết là

$$\min z = \sum_{i=1}^N R_i + \sum_{j=1}^N C_j.$$

Vì luôn có $\sum R_i = \sum C_j$, việc tối thiểu hóa biểu thức trên tương đương với tối thiểu hóa tổng phần tử của B , chỉ khác một hệ số hằng.

So với mô hình ban đầu, số biến giảm xuống còn $O(N)$. Tuy nhiên, còn một câu hỏi: có chắc mọi nghiệm của mô hình tổng hàng/tổng cột đều tương ứng với một ma trận B hay không?

Với nhiệm vụ một, do các phần tử của B có thể nhận giá trị tùy ý, nghiệm tương ứng luôn tồn tại.

Với nhiệm vụ hai, xét các ràng buộc hiện tại:

$$\sum_{i=1}^N R_i = \sum_{j=1}^N C_j, \quad R_i \geq 0, \quad C_j \geq 0.$$

Lấy tùy ý một R_i , rồi rút một phần giá trị từ các C_j để đưa vào hàng i . Do $\sum_j C_j \geq R_i$, luôn tồn tại cách rút sao cho sau đó mọi C_j vẫn không âm. Sau khi xử lý xong R_i , bài toán còn lại có dạng ràng buộc không đổi. Vì vậy chắc chắn tồn tại một ma trận không âm B tương ứng với nghiệm của mô hình trên.

4.2.3 Task1

Khi cả hai nhiệm vụ đều có $N \leq 50$, có thể dùng đơn hình hoặc thuật toán tổng quát tương tự để giải trực tiếp quy hoạch tuyến tính.

Với nhiệm vụ hai, mô hình đúng là dạng chuẩn của quy hoạch tuyến tính. Với nhiệm vụ một, cần tách mỗi R_i thành $R_i^a - R_i^b$, trong đó $R_i^a, R_i^b \geq 0$; với C_j làm tương tự. Trên dữ liệu ngẫu nhiên, đơn hình chạy rất tốt, thậm chí có thể nhanh hơn lời giải chính. Nếu hiện thực tốt, ví dụ thêm cắt tĩa bằng danh sách liên kết, cách này cũng có thể qua được dữ liệu của Task3. Độ phức tạp thời gian: không xác định.

4.2.4 Nguyên lý đối ngẫu

Quy hoạch tuyến tính gốc dường như không dễ xử lý tiếp, nên ta thử chuyển sang bài toán đối ngẫu.

Nhiệm vụ một. Có thể dự đoán rằng trị tuyệt đối của các phần tử trong ma trận đáp án không quá lớn. Nếu cộng cho mỗi phần tử một hằng số rất lớn, rồi dùng kết luận của Task0, nhiệm vụ một sẽ được chuyển thành nhiệm vụ hai. Dĩ nhiên làm trực tiếp như vậy chắc chắn sẽ quá thời gian, nên ta dùng thẳng nguyên lý đối ngẫu.

Bài toán đối ngẫu thu được có dạng

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + P = 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + P = 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} - P = 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} - P = 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} - P = 1, \\ Y_{i,j} \geq 0, \end{cases} \quad \max z = 0 \cdot P + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Nếu không có biến P , đây là mô hình luồng chi phí lớn nhất rất quen thuộc trên đồ thị hai phía. Chú ý rằng mọi ràng buộc đều lấy dấu bằng. Điều này có nghĩa là trong mạng chi phí đó, tổng dung lượng các cạnh rời khỏi S bằng tổng dung lượng các cạnh đi vào T ; từ đó suy ra ngay $P = 0$. Phần còn lại chỉ là một bài toán luồng chi phí đơn giản.

Nhiệm vụ hai, cách 1. Vẫn đối ngẫu theo cách trên, ta nhận được

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + P \leq 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + P \leq 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} - P \leq 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} - P \leq 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} - P \leq 1, \\ Y_{i,j} \geq 0, \end{cases} \quad \max z = 0 \cdot P + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Vì $Y_{i,j} \geq 0$, có thể thấy $P \in [-1, 1]$. Ý nghĩa của mô hình này thực chất là luồng chi phí lớn nhất trên đồ thị hai phía đầy đủ, chỉ khác ở chỗ ta có thể tăng giới hạn lượng ra của mọi đỉnh ở một phía thêm P , đồng thời giảm giới hạn lượng vào của mọi đỉnh phía còn lại đi P .

Trực giác cho thấy, khi P nằm trong các đoạn $[-1, 0]$ và $[0, 1]$, đáp án là một hàm đơn đỉnh. Tác giả ghi rằng việc chứng minh vẫn còn khó, nên trước hết thử lập bảng giá trị; kết quả cho thấy đúng là hàm đơn đỉnh. Gọi hàm đó là $f(P)$. Nó có một vài tính chất sau:

- Chỉ khi $f'(P) = 0$ mới có nhiều giá trị P cho cùng một giá trị $f(P)$. Vì vậy có thể tam phân hàm này.
- Với hàm đơn đỉnh trên $[-1, 1]$, chỉ cần thực hiện một lần tam phân.

Cách này giải được Task4.

Nhiệm vụ hai, cách 2. Đây là ý tưởng ban đầu của tác giả. Tuy độ phức tạp cao hơn, tư tưởng của nó khá khéo.

Nhận xét rằng đáp án của bài toán gốc có tính đơn điệu rõ ràng theo giá trị cần kiểm tra. Vì thế có thể nhị phân trực tiếp đáp án; giả sử đáp án đang thử là K . Khi đó ràng buộc ban đầu

$$\sum_{i=1}^N R_i - \sum_{j=1}^N C_j = 0$$

được thay bằng

$$\sum_{i=1}^N R_i = K, \quad \sum_{j=1}^N C_j = K.$$

Đối ngẫu tiếp, ta được

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + A' \leq 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + A' \leq 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} + B' \leq 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} + B' \leq 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} + B' \leq 1, \\ Y_{i,j} \geq 0, \end{cases}$$

$$\max z = KA' + KB' + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Mô hình luồng của quy hoạch tuyến tính này cũng rất rõ: có thể dùng chi phí K để nối giới hạn lưu lượng của mọi đỉnh ở một phía thêm K . Theo tính chất của đối ngẫu, khi và chỉ khi mô hình này có nghiệm cực đại, bài toán gốc không có nghiệm, tức là giá trị K đang nhị phân nhỏ hơn đáp án đúng.

Rõ ràng khi mô hình này đạt nghiệm cực đại thì phải có $A' \rightarrow -\infty$ và $B' \rightarrow -\infty$. Nhưng ta không thể trực tiếp tìm A' và B' trên khoảng $(-\infty, 1]$. Có thể dự đoán rằng khi A' và B' đủ nhỏ, với một A' bất kỳ, cứ giảm A' đi ΔA thì lượng B' cần giảm, ΔB , sẽ tiến tới một hằng số.

Giả sử lúc này đã biết A' và B' . Nếu $\Delta A \geq \Delta B$, đặt $\Delta A = 1$, rồi tìm ΔB tương đối với ΔA . Rõ ràng ΔB nằm trong đoạn $[0, 1]$ và có thể tam phân: nếu ΔB quá nhỏ thì ΔA còn dư; nếu ΔB quá lớn thì ΔA không đủ bù chi phí $-\Delta B \cdot K$. Trường hợp $\Delta A \leq \Delta B$ làm tương tự bằng cách đặt $\Delta B = 1$.

Còn cần xác định A' và B' ban đầu. Vì A'/B' cuối cùng sẽ tiến tới $\Delta A/\Delta B$, chỉ cần thay $\Delta A = 1$ bằng $\Delta A = P_0$, với P_0 đủ lớn. Khi K càng gần đáp án, độ chính xác cần thiết của tỉ số A'/B' càng cao; nếu muốn nhận được chính xác đáp án K , P_0 phải rất lớn. Tuy nhiên đáp án chỉ cần 4 chữ số sau dấu thập phân, nên không cần chính xác đến vậy; P_0 có thể chọn tương đối nhỏ. Qua thử nghiệm, trong tình huống thông thường, lấy $P_0 = N^2$ là đủ để đạt nghiệm tối ưu.

So với thuật toán thứ nhất, cách này có thêm một lớp nhị phân, nên độ phức tạp tăng thêm một thừa số log. Đáng tiếc là nó chỉ vừa đủ qua dữ liệu $N \leq 50$, tức Task3.

4.2.5 Task2

Trong nhiệm vụ hai, với phần lớn dữ liệu ngẫu nhiên, $P = 0$ đã là nghiệm tối ưu. Chỉ có rất ít bộ dữ liệu làm P dao động trong khoảng $[-10^{-4}, 10^{-4}]$, về cơ bản có thể bỏ qua. Do đó có thể xuất đáp án của nhiệm vụ một cho câu hỏi thứ hai.

4.2.6 Khác

Nếu ma trận có kích thước $N \times M$ thì đương nhiên cũng có thể làm tương tự. Khi đó với nhiệm vụ một, P không còn bằng 0, mà bằng

$$P = \frac{M - N}{N + M}.$$

Bài này là một đồ thị hai phía đầy đủ. Nếu dùng SPFA để chạy luồng chi phí thì hiệu quả khá thấp; cần dùng zkw hoặc nguyên thủy-đối ngẫu.

Đơn hình trong nhiều trường hợp cũng có hiệu suất không tệ, gần với zkw luồng chi phí. Nhưng nếu người ra dữ liệu tận dụng đủ tri thức của mình thì vẫn có thể làm nó bị kẹt.

Đồ thị hai phía của nhiệm vụ hai có tính chất đặc biệt nhất định. Nếu có thể bỏ được bước tam phân, hoặc dùng cách khác để tối ưu mỗi lần chạy luồng chi phí, tác giả rất hoan nghênh trao đổi thêm.

4.3 Tổng kết

Bài này có nhiều phiên bản dẫn xuất. Tuy nhiên các phiên bản khác hoặc có đề bài quá phức tạp, hoặc tác giả chưa tìm được thuật toán đủ tốt. Nếu có ý tưởng tốt hơn cho lớp bài này, tác giả rất mong được liên hệ.

Cảm hứng của bài đến từ một bài trước đây của tác giả tên là *Chessboard*. Mô hình ban đầu là đặt quân xe trên bàn cờ, yêu cầu mỗi ô chỉ được đặt nhiều nhất một quân, và ô (i, j) cần được ít nhất $W(i, j)$ quân xe khống chế. Mô hình này tuy mô tả đơn giản, nhưng dường như là NP-khó: nó yêu cầu nghiệm nguyên của một quy hoạch tuyến tính, trong khi nghiệm tối ưu của quy hoạch tuyến tính này hầu như đều là số thực. Sau nhiều lần làm yếu ràng buộc, tác giả cuối cùng thu được bài toán hiện tại.

Bài toán kiểm tra kiến thức cơ bản của thí sinh về quy hoạch tuyến tính và mô hình hóa luồng mạng, đồng thời dùng nguyên lý đối ngẫu. Tính chất của thuật toán thứ nhất rất đẹp, dù phần chứng minh vẫn còn để ngỏ trong bài gốc. Với thuật toán thứ hai, ta lồng nhị phân và tam phân, rồi dùng luồng chi phí để phán định có nghiệm hay không. Tác giả cho rằng đây là một tư tưởng khá hay: chuyển bài toán tối ưu hóa thành bài toán phán định tính khả thi, rồi hiện thực nó trên mạng luồng. Tiếc rằng với bài này, đó không phải thuật toán tốt nhất.

Hy vọng bài toán giúp độc giả có thêm gợi mở và hiểu sâu hơn về quy hoạch tuyến tính cũng như các bài toán luồng mạng.

4.4 Lời cảm ơn

Cảm ơn CCF đã cung cấp nền tảng học tập và trao đổi. Cảm ơn thầy Xiang Qizhong đã hướng dẫn tác giả. Cảm ơn các bạn học đã giúp đỡ tác giả.

5 Đa giác biến đổi

Dong Honghua, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài viết tổng kết các phép biến đổi thường dùng với đa giác, cho thấy tính đa dạng của cách nhìn một đa giác, đồng thời minh họa tác dụng của từng phép biến đổi qua ví dụ. Những biến đổi này cung cấp hướng suy nghĩ cho các bài toán liên quan tới đa giác. Bài viết cũng đi sâu vào một dạng biến đổi để nghiên cứu bài toán thống kê thông tin điểm bên trong đa giác, từ đó thu được một thuật toán có độ phức tạp khá tốt.

5.1 Dẫn nhập

Những năm gần đây, đa giác xuất hiện thường xuyên trong các kỳ chọn đội tuyển cấp tỉnh, các cuộc thi OI, cũng như ACM World Finals. Hình thức xuất hiện của chúng phức tạp và thay đổi đa dạng.

Tác giả tổng kết một số cách xử lý như sau. Phần 2 giới thiệu vài kiến thức cơ bản sẽ dùng về sau. Phần 3 liệt kê nhiều phép biến đổi của đa giác, kèm ví dụ để người đọc dễ hiểu và mở rộng. Phần 4 xuất phát từ GIS, tức hệ thống thông tin địa lý, để dẫn vào bài toán thống kê thông tin điểm bên trong đa giác; bài toán được thảo luận từ trường hợp đặc biệt tới tổng quát, từ dễ tới khó, và cuối cùng cho một thuật toán tổng quát có độ phức tạp tốt.

5.2 Cơ sở

5.2.1 Định nghĩa đa giác

Đa giác. Một hình phẳng khép kín tạo bởi ít nhất ba đoạn thẳng nối đuôi nhau theo thứ tự.

Đa giác đơn. Một đa giác mà các cạnh không kề nhau không giao nhau.

Đa giác lồi. Một đa giác đơn mà mọi góc trong đều không vượt quá 180° .

5.2.2 Tích có hướng

Với hai vector hai chiều $(x_1, y_1), (x_2, y_2)$, tích có hướng của chúng là

$$x_1y_2 - x_2y_1.$$

Giá trị này bằng diện tích có hướng của hình bình hành do hai vector tạo thành.

5.2.3 Phương pháp tia

Phương pháp tia dùng để phán định một điểm có nằm trong đa giác hay không. Từ điểm hiện tại, kẻ một tia theo một hướng nào đó rồi đếm số giao điểm giữa đa giác và tia này. Nếu số giao điểm là lẻ thì điểm nằm trong đa giác; nếu là chẵn thì điểm nằm ngoài.

Để tiện tính toán, có thể chọn tia hướng lên trên và hơi lệch sang phải, nhằm tránh trường hợp giao điểm rơi đúng vào đỉnh đa giác. Phương pháp này cũng áp dụng được cho đa giác tự cắt.

Ví dụ 1: Bean. Nguồn: [bzoi 1294] SCOI2009 Bean.

Đại ý đề bài. Cho $D \leq 9$ điểm có trọng số trên một lưới $n \times m$. Hãy tìm một đường đi xuất phát từ một điểm rồi quay lại điểm đó sao cho tổng trọng số các điểm bị bao quanh trừ đi số bước là lớn nhất.

Phân tích. Đường đi tạo thành một đa giác có thể tự cắt. Vì vậy việc phán định mỗi điểm có trọng số có bị bao quanh hay không trở thành bài toán điểm nằm trong đa giác.

Dùng phương pháp tia: với mỗi điểm có trọng số, kẻ tia hướng lên trên; khi đi qua một cạnh giao với tia đó thì trạng thái của điểm được lật. Dùng nhị phân để ghi trạng thái của từng điểm có trọng số, khi đó tổng giá trị điểm có thể suy ra từ trạng thái cuối.

Do đó chỉ cần liệt kê điểm xuất phát, tính đường ngắn nhất để đạt tới từng trạng thái nhị phân rồi quay lại điểm xuất phát. Vì trọng số cạnh đều bằng 1, có thể dùng bfs.

5.3 Các phép biến đổi

5.3.1 Kết hợp với gốc tọa độ

Cạnh là thành phần chính của đa giác. Nếu kết hợp mỗi cạnh với gốc tọa độ, ta thu được một số tam giác.

Theo cách phán định của phương pháp tia, có thể chứng minh diện tích đa giác bằng tổng diện tích có dấu của các tam giác này. Diện tích mỗi tam giác có thể tính trực tiếp bằng tích có hướng. Cần chú ý thứ tự lấy tích có hướng khác nhau sẽ cho dấu diện tích khác nhau; điều này vừa đúng lúc phản ánh chiều đi quanh của đa giác.

Ví dụ 2: Pollution Solution. Nguồn: *World Finals 2013 Problem J.*

Đại ý đề bài. Tính diện tích phần giao giữa một đa giác và một hình tròn.

Phân tích. Diện tích đa giác có thể tách thành một số diện tích có hướng của tam giác, nên bài toán rút gọn thành tính diện tích phần giao giữa tam giác và hình tròn. Nếu tách theo tâm đường tròn, ta bảo đảm một đỉnh của tam giác nằm ở tâm. Sau đó chia bốn trường hợp theo quan hệ vị trí giữa tam giác và đường tròn rồi tính riêng từng trường hợp.

5.3.2 Tách theo trục tọa độ

Tương tự, có thể kết hợp cạnh với trục tọa độ: trừ các cạnh thẳng đứng, mỗi cạnh được chiếu xuống một nửa trục, và cạnh cùng hình chiếu của nó tạo thành một hình thang vuông hoặc hình chữ nhật.

5.3.3 Nửa mặt phẳng

Một đa giác lồi có thể xem là giao của một số nửa mặt phẳng.

Ví dụ 3: Giao đa giác lồi. Nguồn: *[bzoj 2618] CQOI2006.*

Đại ý đề bài. Tính diện tích giao của n đa giác lồi.

Phân tích. Giao của các đa giác lồi chính là giao của toàn bộ các nửa mặt phẳng tương ứng.

Ở đây giới thiệu một thuật toán giao nửa mặt phẳng dùng phương pháp tăng dần, độ phức tạp bậc hai nhưng dễ viết. Với mỗi nửa mặt phẳng mới, tìm các điểm của giao hiện tại nằm bên trong và bên ngoài nửa mặt phẳng đó; xóa các điểm bên ngoài, đồng thời thêm giao điểm giữa nửa mặt phẳng mới và các đoạn nối một điểm trong với một điểm ngoài. Như vậy ta thu được giao mới.

5.3.4 Cây

Mọi đường chéo của đa giác lồi đều nằm bên trong nó, nên một đường chéo có thể chia đa giác lồi thành hai đa giác lồi. Làm đệ quy sẽ thu được một phép tam giác hóa. Các tam giác được tách ra tạo thành một cây.

Đa giác đơn cũng có thể biến thành cây. Từ mọi đỉnh, kéo dài theo phương ngang sang trái và phải cho tới giao điểm đầu tiên; kéo dài thành đường thẳng cũng được. Cách này chia đa giác thành nhiều hình thang, và các hình thang đó tạo thành một cây.

Ví dụ 4: Journey. Nguồn: *[bzoj 2657] ZJOI2012 journey.*

Đại ý đề bài. Cho phép tam giác hóa của một đa giác lồi. Hãy tìm một đường chéo sao cho số tam giác mà nó đi qua là lớn nhất.

Phân tích. Phép tam giác hóa tạo thành cấu trúc cây, nên thử biến đường chéo thành một đường đi trên cây.

Kết luận là: các đường chéo của đa giác lồi có số tam giác đi qua khác 0 tương ứng một-một với mọi đường đi trên cây.

Khi $n = 3$, kết luận hiển nhiên đúng. Giả sử kết luận đúng với $n = k$, ta chứng minh với $n = k + 1$. Xóa một đỉnh của đa giác lồi $k + 1$ cạnh thì thu được đa giác lồi k cạnh; trên cây chỉ cần xét trường hợp thêm một nút lá x , có cha là y . Theo giả thiết quy nạp, các đường chéo cũ tương ứng với các đường đi trong cây cũ. Khi thêm đỉnh bị xóa trở lại, vì y là cha của x , mọi đoạn nối từ đỉnh mới trong vùng x tới các đỉnh khác đều phải đi qua cạnh nối giữa x và y . Vì vậy các đoạn đó tương ứng với mọi đường đi trên cây xuất phát từ x . Kết luận đúng với $n = k + 1$.

Như vậy, tìm đường chéo đi qua nhiều tam giác nhất trở thành tìm đường dài nhất trên cây; có thể giải bằng quy hoạch động trên cây hoặc hai lần bfs.

Ví dụ 5: *Toil for Oil.* Nguồn: *World Finals 2002 Problem F.*

Đại ý đề bài. Cho một đa giác có lỗ trên biên. Hãy tính diện tích phần bên trong đa giác có thể chứa dầu.

Phân tích. Dùng các đường thẳng nằm ngang qua mọi đỉnh để cắt đa giác thành nhiều hình thang. Trong mỗi hình thang, hoặc toàn bộ có dầu, hoặc hoàn toàn không có dầu.

Quét từ dưới lên trên để sinh từng vùng. Có thể dùng đường trung tuyến của dải hiện tại để cắt: nếu nó giao với cạnh đa giác thì thêm biên trên và biên dưới; sắp xếp hai dãy biên này sẽ thu được quan hệ tương ứng. Nếu phía dưới thành phần liên thông hiện tại đã có lỗ, phần phía trên không thể có dầu, nên đánh dấu thành phần liên thông đó là không khả thi. Phần phía trên không ảnh hưởng tới dải hiện tại, nên trực tiếp cộng diện tích khả thi trong dải hiện tại vào đáp án.

Dùng DSU để duy trì tính liên thông của cây, đồng thời dùng các lỗ trên biên dưới trước khi thống kê dải hiện tại và trên biên trên sau khi thống kê để cập nhật trạng thái của thành phần liên thông.

5.4 Loạt *Architect*: thống kê thông tin điểm trong đa giác

5.4.1 Ứng dụng trong GIS

Trong GIS, hệ thống thông tin địa lý, ta thường cần thống kê các giá trị đặc trưng nằm trong một vùng. Vùng thường có thể biểu diễn bằng đa giác, nên các bài toán này trở thành bài toán thống kê thông tin điểm bên trong đa giác.

Trong phần lớn hệ thống GIS, cách làm là dùng phương pháp tia để lần lượt phán định từng điểm có nằm trong đa giác hay không. Thuật toán này có hiệu suất thấp và chưa tận dụng tốt các tính chất của bản đồ.

5.4.2 *Architect*

Đại ý đề bài. Nguồn: *POJ Challenge Round 5, lời mời của Trường Trung học số 1 Thiệu Hưng.*

Hỗ trợ thêm điểm động và hỏi số điểm nằm trong một đa giác đơn mà mọi góc trong đều là bội của 45° . Các điểm nằm ở tâm ô lưới, còn đa giác nằm trên biên ô lưới.

Vết cặn. Dùng phương pháp tia để lần lượt phán định từng điểm có nằm bên trong hay không.

Tách đa giác. Vì đa giác khá phức tạp, ta xét cách tách. Nếu đem phương pháp dùng gốc tọa độ và từng cạnh để tạo tam giác, vốn dùng khi tính diện tích đa giác, áp dụng vào bài này, ta sẽ sinh ra một số cạnh vừa không song song với trục tọa độ, vừa không tạo góc 45° ; điều này phá hỏng điều kiện của đề.

Vì vậy xét một cách tách khác. Trừ cạnh thẳng đứng, mỗi cạnh được chiếu xuống trục x ; cạnh đó và hình chiếu tạo thành hình thang vuông 45° hoặc hình chữ nhật. Số điểm trong toàn bộ các hình thang vuông 45° và hình chữ nhật, cộng trừ theo hướng duyệt, sẽ cho số điểm trong đa giác ban đầu: nếu duyệt theo chiều kim đồng hồ thì cạnh hướng sang phải lấy dấu $+$, cạnh hướng sang trái lấy dấu $-$. Cần chú ý rằng điểm nằm trên biên đa giác cũng được tính, nên hình thang vuông 45° mang dấu $-$ phải dịch xuống một ô.

Chuyển hóa bài toán. Sau khi tách đa giác, bài toán trở thành đếm số điểm trong các hình thang vuông 45° và hình chữ nhật.

Trước hết bỏ qua thao tác thêm điểm động; giả sử mọi điểm đã được thêm xong rồi mới hỏi. Với truy vấn số điểm trong một hình chữ nhật có một cạnh nằm trên trục x , có thể chuyển nó thành: phần đỏ cộng phần xanh, trừ phần xanh. Cả hai phần đều là truy vấn số điểm nằm phía dưới bên trái một điểm truy vấn.

Sau đó dùng thuật toán quét: sắp xếp thao tác thêm điểm và truy vấn theo tọa độ x , dùng tọa độ y làm khóa phụ; khi quét tới một truy vấn, đưa mọi điểm ở bên trái điểm truy vấn vào cấu trúc dữ liệu, chẳng hạn cây Fenwick, rồi theo tọa độ y để hỏi tổng đoạn hoặc tổng tiền tố trong cấu trúc dữ liệu.

Với hình thang vuông 45° , cũng dùng cách tương tự; lúc này khóa y được thay bằng $x + y$.

Chia để trị cdq. Cuối cùng, do đề bài cho thêm điểm và hỏi xen kẽ, còn đóng góp của điểm cho mỗi truy vấn là độc lập, có thể chia để trị theo thời gian, tức cdq. Tất cả sự kiện ban đầu được sắp theo thời gian; chọn trung điểm, chia dãy thành hai nửa, đệ quy trái và phải, rồi tính đóng góp của các điểm bên trái cho các truy vấn bên phải. Vì thời gian của bên trái đều nhỏ hơn bên phải, phần này trở lại trường hợp thêm điểm trước rồi mới hỏi, dùng phương pháp vừa nêu là được.

Độ phức tạp thời gian.

$$O(n \log^2 n).$$

5.4.3 Architekt1

Đại ý đề bài. Nguồn: lần hỏi thứ nhất của kỳ hồ tuyển đội tuyển 2014.

Ban đầu có n điểm trên mặt phẳng, mỗi điểm có trọng số. Có Q thao tác; mỗi thao tác là sửa trọng số điểm hoặc hỏi tổng trọng số các điểm nằm trong một đa giác đơn. Bảo đảm các cạnh của mọi đa giác truy vấn tạo thành một đồ thị phẳng liên thông.

Không có sửa đổi. Trước hết xét trường hợp không có sửa đổi. Tổng trọng số điểm có thể cộng trừ. Tách đa giác theo trục tọa độ: mỗi cạnh và hình chiếu của nó lên trục x tạo thành một hình thang. Ta cần tính tổng trọng số trong hình thang và cộng trừ theo hướng cạnh khi thống kê.

Chuyển sang cách nhìn “điểm đóng góp cho hình thang”. Dùng đường quét để tối ưu: dùng cây cân bằng duy trì các đoạn thẳng còn giao với đường quét; khi gặp một điểm ứng viên, cộng một giá trị cho mọi đoạn nằm phía trên nó. Cuối cùng thống kê tổng trọng số trên mọi đoạn và cộng trừ theo hướng.

Độ phức tạp thời gian.

$$O(S \log S),$$

trong đó S là tổng quy mô cạnh cần xử lý.

Chia để trị cdq. Nếu dùng thuật toán trên mà cần tăng hoặc giảm trọng số điểm thì làm thế nào?

Phần cộng trừ vẫn có thể tách được, nên có thể lồng cdq: biến tình huống sửa và hỏi xen kẽ thành sửa trước rồi hỏi sau. Cụ thể, với dãy hiện tại, chọn trung điểm, tính ảnh hưởng của các sửa đổi bên trái lên các truy vấn bên phải; phần này chính là trường hợp sửa trước hỏi sau, quay về bài toán trên. Sau đó đệ quy xử lý riêng nửa trái và nửa phải.

Độ phức tạp thời gian.

$$O(S \log S \log Q).$$

5.4.4 Archtext2

Đại ý đề bài. Nguồn: kỳ hồ tuyển đội tuyển 2014.

Trên cơ sở bài trước, ngoài tổng trọng số điểm còn phải hỏi trọng số điểm lớn nhất.

Chuyển hóa bài toán. Vì đã cho đồ thị phẳng, ta thử tìm cấu trúc trên đồ thị đối ngẫu. Trong đồ thị đối ngẫu, mỗi đa giác truy vấn bao quanh một thành phần liên thông, và thành phần đó cũng là liên thông trên đồ thị đối ngẫu. Mỗi cạnh của đa giác chính là một cạnh trên đồ thị đối ngẫu. Vì vậy bài toán chuyển thành: xóa một số cạnh rồi hỏi thông tin trong một thành phần liên thông. Khi đa giác suy biến thành đoạn thẳng thì bên trong không có miền nào; trường hợp này có thể đặc biệt xử lý trực tiếp.

Tiếp theo dẫn vào bài toán đồ thị động.

Bài toán đồ thị động. Mô tả ngắn gọn: nhiều lần hỏi thông tin của một thành phần liên thông sau khi xóa một số cạnh.

Dùng cdq. Tại mọi thời điểm, bảo đảm mọi cạnh không xuất hiện trong bất kỳ truy vấn nào của đoạn hiện tại đều đã được gộp. Đồng thời ghi số lần mỗi cạnh xuất hiện trong các truy vấn của đoạn.

Khi đệ quy sang nửa trái, thêm vào đồ thị các cạnh xuất hiện ở truy vấn nửa phải nhưng không xuất hiện ở truy vấn nửa trái, rồi dùng DSU để duy trì thông tin trong thành phần liên thông. Sau khi đệ quy trở về, cần khôi phục DSU về trạng thái trước khi đệ quy, tức xóa các cạnh vừa thêm do nửa trái.

Vì những cạnh bị xóa là các cạnh được thêm sau cùng, chỉ cần thao tác trên ngăn xếp thêm cạnh: hoàn tác thao tác gán cha trong DSU và khôi phục thông tin được duy trì. Khi đệ quy sang nửa phải cũng làm tương tự và khi quay lại cũng khôi phục.

Khi đoạn chỉ còn một truy vấn, trực tiếp lấy thông tin của thành phần liên thông chứa nó.

Tương ứng thông tin. Khó khăn còn lại là đưa thông tin điểm trong đồ thị phẳng ban đầu lên các đỉnh của đồ thị đối ngẫu, tức các miền của đồ thị phẳng.

Một truy vấn là phần bên trong của một đa giác. Cần tìm ít nhất một miền nằm bên trong, chẳng hạn miền ở bên phải một cạnh khi duyệt đa giác theo chiều kim đồng hồ, rồi hỏi thông tin của thành phần liên thông chứa miền đó.

Không có sửa đổi. Khi tính max, một điểm có thể bị tính lặp nhiều lần. Nếu không có sửa đổi, cứ lấy trọng số lớn nhất của mọi điểm thuộc một miền làm trọng số của miền đó, rồi áp dụng thuật toán đồ thị động để hỏi trọng số điểm lớn nhất trong một thành phần liên thông.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Số điểm. Xét tiếp trường hợp đặc biệt mọi trọng số điểm đều bằng 1, tức cần hỏi số điểm bên trong. Có thể dùng công thức Euler trên đồ thị phẳng liên thông:

$$\text{số điểm} = \text{số cạnh} - \text{số miền} - 1.$$

Số miền và số cạnh trong một thành phần liên thông có thể tính tương đối dễ, từ đó suy ra số điểm.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Trường hợp tổng quát. Với trường hợp tổng quát, ta cần ánh xạ mỗi điểm ban đầu vào một miền nào đó.

Tiếp tục phân tích truy vấn, có thể thấy lân cận của mọi điểm nằm bên trong đa giác cũng nằm trong thành phần liên thông tương ứng trên đồ thị đối ngẫu. Vì vậy, với điểm bên trong, chọn tùy ý một miền làm miền tương ứng là được.

Điểm trên biên không thỏa tính chất này, nhưng các điểm đó có thể lấy trực tiếp từ thông tin của truy vấn. Ngoài ra cần cưỡng chế rằng điểm trên biên không được thêm vào miền khi xử lý truy vấn; trong cdq, phải ghi thêm thông tin về số lần một điểm xuất hiện. Cụ thể, khi đệ quy nửa trái, cần đưa các điểm xuất hiện ở nửa phải nhưng không xuất hiện ở nửa trái vào miền tương ứng của chúng.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Thêm sửa đổi. Khi có thao tác sửa đổi, do ta đã có thông tin về số thao tác mà một điểm liên quan tới, chỉ cần tính cả sửa đổi vào đó. Khi đệ quy tới đoạn chỉ còn thao tác sửa đổi thì thực hiện sửa đổi.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

5.4.5 ArchiEXT

Đại ý đề bài. Trên cơ sở bài trước, không cho trước đồ thị phẳng liên thông tạo bởi toàn bộ đa giác truy vấn.

Tự lực xây dựng. Không cho đồ thị phẳng nhưng vẫn muốn dùng thuật toán trên, nên chỉ còn cách tự xây dựng đồ thị phẳng. Dùng đường quét để tìm mọi giao điểm giữa các cạnh của các đa giác; các giao điểm này cùng với các cạnh đa giác tạo thành đồ thị phẳng.

Bước này có độ phức tạp

$$O(\text{số giao điểm} \cdot \log S).$$

Không liên thông. Cần chú ý đồ thị phẳng thu được bằng cách này không nhất thiết liên thông. Vì vậy phải làm cho nó liên thông. Áp dụng phương pháp tác giả trình bày trong buổi trao đổi học viên WC2014, có thể chuyển nó thành đồ thị phẳng liên thông. Sau khi có đồ thị phẳng liên thông, bài toán trở thành bài trước.

Tối ưu. Điểm nghẽn của độ phức tạp là bước tìm toàn bộ giao điểm, tức $O(\text{số giao điểm})$. Tìm hết mọi giao điểm vừa không hợp lý lắm, vừa quá lãng phí.

Xét chia khối: cứ mỗi T truy vấn tạo thành một khối. Trong khối này cần dựng đồ thị phẳng; những điểm còn lại dùng định vị điểm bằng đường quét để đưa vào một miền. Khi đó dựng đồ thị phẳng tốn

$$O\left(\frac{S}{T}T^2 \log S\right),$$

định vị các điểm còn lại tốn

$$O\left(\frac{S}{T}S \log S\right),$$

và giải T truy vấn tốn

$$O\left(\frac{S}{T}T \log^2 T\right).$$

Tổng hợp lại, chọn $T = \sqrt{S}$ là tối ưu, cho độ phức tạp cuối cùng

$$O(S^{1.5} \log S).$$

Vẫn còn một vấn đề: nếu một đa giác truy vấn có quá nhiều cạnh, làm đây không thể chia theo $\sqrt{S}$ thì sao? Chú ý số đa giác như vậy không nhiều, và bên trong chỉ có một miền, nên có thể dùng trực tiếp định vị điểm để phán định, với tổng độ phức tạp $O((nQ + S) \log S)$. Lấy riêng mọi truy vấn có số cạnh lớn hơn $\sqrt{S}$, số lượng không vượt quá $\sqrt{S}$, rồi giải từng truy vấn độc lập; như vậy vẫn bảo đảm độ phức tạp $O(S^{1.5} \log S)$.

5.4.6 Tiểu kết

Khi các thuật toán thường dùng gặp khó ở truy vấn max, tác giả chọn một hướng khác: ghép nhiều đa giác thành một đồ thị phẳng rồi chuyển bài toán thành bài toán đồ thị.

Trong bài toán này còn nhiều lần dùng chia để trị theo thời gian, tức cdq, để biến phức tạp thành đơn giản. Ở cuối, bài viết dùng thêm tư tưởng chia khối và phân loại trường hợp để giảm độ phức tạp xấu nhất, từ đó thu được một thuật toán khá tốt.

5.5 Tổng kết

Mặc dù số cạnh của đa giác không cố định và hình dạng phức tạp, gây nhiều khó khăn khi giải bài, nhưng thông qua những cách tách khéo léo, đa giác có thể biến thành một số phần nhỏ hơn, tạo điều kiện để xử lý từng phần.

Sau khi phân tích đặc điểm của đa giác trong đề, áp dụng các phương pháp tách thường dùng thường đem lại hiệu quả tốt.

5.6 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Cảm ơn các thầy Chen Heli, Shao Hongxiang, You Guanghui và Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã quan tâm và hướng dẫn tác giả trong nhiều năm. Cảm ơn huấn luyện viên đội tuyển quốc gia Hu Weidong và Yu Linyun đã hướng dẫn. Cảm ơn các anh chị Xu Jie, Zhong Yiqin, Fei Jiezhong, Liang Jiawen của Đại học Thanh Hoa và Jiang Youzhi của Đại học Giao thông Thượng Hải đã giúp đỡ tác giả. Cảm ơn các bạn He Qizheng, Zheng Zhongyi, Xu Zhengjie của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở. Cảm ơn các bạn Yu Yili, Zhang Hengjie, Wang Jianhao, Guo Yu và nhiều bạn học khác của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở. Cảm ơn tất cả thầy cô và bạn bè từng giúp đỡ tác giả. Cảm ơn cha mẹ và gia đình đã ủng hộ, quan tâm và chăm sóc chu đáo.

Tài liệu tham khảo

1. SUN, “Bàn về một lớp thuật toán chia để trị”, bài giảng WC2013.
2. RRR, “Bàn về vài cách giải phi kinh điển cho bài cấu trúc dữ liệu”, luận văn đội tuyển quốc gia 2013.
3. HA, “Ứng dụng đường quét trong hình học tính toán”, trao đổi học viên WC2014.
4. 246, “Báo cáo ra đề *Architext*”, bài tập đội tuyển quốc gia 2014.

Ghi chú về nguồn

Tệp PDF có 228 trang. pdftotext đọc tốt phần mục lục nhưng phần thân chủ yếu trở thành ký tự mojibake do ánh xạ phonetic. Bài *MSS* được dịch từ các trang in 1–8 (trang vật lý PDF 5–12). Bài *Matrix* được dịch từ các trang in 9–18 (trang vật lý PDF 13–22); trang in 18 là trang trắng ngăn cách, nội dung bài nằm trên các trang in 9–17. Bài *Đa giác biến đổi* được dịch từ các trang in 19–34 (trang vật lý PDF 23–38); trang in 34 là trang trắng ngăn cách, nội dung bài nằm trên các trang in 19–33. Các bài đã dịch đều dùng các trang render bằng pdftoppm và OCR bằng tesseract. Một số ký hiệu trong phần ràng buộc nhóm D của bài *MSS* bị nhiễu, nên bản dịch ghi rõ phần suy ra từ lời giải thay vì chép một công thức không chắc chắn. Ở bài *Matrix*, các công thức đối ngẫu được đối chiếu giữa OCR, lớp văn bản mojibake còn đọc được ký hiệu và ảnh render của các trang nguồn. Ở bài *Đa giác biến đổi*, các hình minh họa trong PDF nguồn được chuyển thành mô tả văn bản; các dòng độ phức tạp bị OCR nhiễu được đối chiếu lại bằng ảnh render.